

Trainer Guide: Spring Boot Session 3 - DTO, Layers & Exception Handling

A slide-by-slide read-aloud script for [springboot-session3-theory.html](#). Everything on screen, explained in plain spoken language.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: [springboot-session3-theory.html](#) (21 slides)

Each slide has an ON SCREEN reminder and a SAY section you can read out word-for-word. EMPHASISE and DELIVERY TIP notes are for you only - do not read those aloud.

Slide 1 - Title: DTO, Layered Architecture & Exception Handling

ON SCREEN

The blue cover slide naming the session, plus 'a first look at Spring Security and Microservices'.

SAY (read aloud)

Welcome to Session 3. In Session 2 we built a working CRUD API. Today we make it professional. We'll separate our application into clean layers, stop exposing our database objects to the outside world by introducing DTOs, and handle errors gracefully so clients get clear messages instead of ugly stack traces. Then we'll close with two forward-looking overviews - a first taste of Spring Security and an introduction to microservices - that frame everything you've built so far in a bigger picture.

Slide 2 - Agenda

ON SCREEN

Six agenda items: layered architecture, DTO, exception handling, Spring Security demo, microservices concepts, recap.

SAY (read aloud)

Here's today's path. First, layered architecture - Controller, Service, Repository, each with one job. Then DTOs - why you never expose your entity directly. Then exception handling - custom exceptions and clean 404s. Then a five-minute Spring Security demo, no lab. Then microservices concepts - the bigger picture. And a recap with an annotation cheat-sheet. The thread today: we take the working-but-rough API from Session 2 and make it the kind of code you'd be comfortable shipping.

Slide 3 - Session 2 recap

ON SCREEN

Three click-to-reveal flash cards: @RequestBody, POST vs PUT, and @PathVariable.

SAY (read aloud)

Three quick warm-up questions - answer in your head, then we reveal. Question one: what does @RequestBody do? It takes the JSON in the request body and deserializes it into a Java object, via Jackson - you put it on a method parameter and Spring hands you a ready-built object. Question two: the difference between POST and PUT? POST creates a new resource - the C in CRUD; PUT updates an existing one, usually identified by an id in the path - the U. Question three: what does @PathVariable do? It binds a segment of the URL path to a method parameter - in slash posts slash id, the id placeholder is bound to the method's argument.

Slide 4 - What's wrong with returning entities directly?

ON SCREEN

Two code panels: returning a User entity that leaks password and internal flags, versus a UserDTO exposing only safe fields.

SAY (read aloud)

Let's start with the problem we're solving. In Session 2 our controller returned the same object that mapped to the database. That's convenient - but it leaks. Look at the left panel: returning the User entity directly sends every field to the client, including the password hash, an isAdmin flag, and a failedLogins count - all internal things the client should never see. The right panel returns a UserDTO instead, exposing only id, name, email, and role. The core problem: your database table and your API response are two different things with two different audiences. Letting one dictate the other couples them together - change a column and you accidentally change, or break, your public API. The fix is a dedicated object for transfer, a DTO.

Slide 5 - What is a DTO?

ON SCREEN

A definition and three reason cards: security, flexibility, decoupling.

SAY (read aloud)

A DTO - Data Transfer Object - is a plain Java class whose only job is to carry data across the boundary between your application and the outside world. No JPA annotations, no business logic - it carries only what the client needs to send or receive. Three reasons make it worth the extra class. Security: hide sensitive fields - passwords and internal flags live on the entity but never appear on the DTO. Flexibility: shape the response to the client's needs - one screen wants a summary, another wants detail. Decoupling: your database schema can change without breaking the API contract - rename a column, map it in the service, and clients never notice.

EMPHASISE

Give them the one-liner to remember: the entity maps to the database; the DTO travels over the wire. They often have similar fields, but they exist for completely different reasons - and keeping them separate is what makes an API safe to evolve.

Slide 6 - Layered architecture

ON SCREEN

A vertical pipeline: Client, Controller, Service, Repository, Database, each labelled with its annotation and responsibility.

SAY (read aloud)

This is the architectural anchor of the whole session. A professional Spring Boot app is split into layers, each with one clear job, and a request flows down through them while the response flows back up. The Client sends an HTTP request. The Controller, marked at-RestController, receives it, validates input, and calls the Service with a DTO. The Service, marked at-Service, holds the business logic and maps between DTO and entity. The Repository, marked at-Repository, talks to the database using the entity. And the Database stores and retrieves the data - H2 today.

EMPHASISE

Stress single responsibility, layer by layer: the controller only deals with HTTP, the service only with business rules, the repository only with the database. No layer reaches past its neighbour - and that discipline is exactly what keeps a growing codebase from turning into spaghetti.

Slide 7 - Entity vs DTO, side by side

ON SCREEN

A comparison table, then two code blocks: Post.java the entity and PostDTO.java the DTO.

SAY (read aloud)

Let's put the two classes side by side. Read the table. Where it lives: the entity is mapped to a database table; the DTO travels between client and service. Annotations: the entity has JPA annotations - at-Entity, at-Id, at-GeneratedValue; the DTO has none of those, just at-Data from Lombok. Exposure: the entity is internal and never sent to clients; the DTO is external-facing. Contents: the entity has all columns, even sensitive ones; the DTO has only what the client needs. Now the code: Post, the entity, carries the JPA annotations and maps to the posts table. PostDTO has the same four fields - id, title, description, content - but no JPA annotations at all. Same shape, different purpose.

Slide 8 - DTO mapping, the conversion

ON SCREEN

Two helper methods, mapToDTO and mapToEntity, and a round-trip flow diagram for incoming and outgoing data.

SAY (read aloud)

Because the controller speaks DTO and the repository speaks entity, the service sits in the middle and converts between them. We write two small helper methods for this - one each way. mapToDTO takes a Post entity and copies its fields into a new PostDTO - that's the way out, to the client. mapToEntity takes a PostDTO and copies its fields into a new Post - that's the way in, from the client. Follow the round-trip on screen. Incoming: JSON becomes a PostDTO via at-RequestBody, the service maps it to a Post entity, and it's saved to the database. Outgoing: a Post from the database is mapped to a PostDTO, which becomes the JSON response. We map by hand today so the conversion is visible; larger projects use libraries like MapStruct to generate these, but the concept is identical.

Slide 9 - Lombok

ON SCREEN

A table of Lombok annotations - `@Data`, `@AllArgsConstructor`, `@NoArgsConstructor`, `@Builder` - and a before/after code size comparison.

SAY (read aloud)

Lombok is a library that generates repetitive Java code for you at compile time. Instead of hand-writing getters, setters, constructors, and `toString` on every class, you add one annotation and Lombok writes them. We use it on both our entity and our DTO. From the table: `@Data` is the all-in-one workhorse - it generates getters, setters, `toString`, `equals`, and `hashCode`.

`@AllArgsConstructor` makes a constructor taking every field. `@NoArgsConstructor` makes an empty one, which JPA and Jackson both need. And `@Builder` gives the builder pattern. The before-and-after shows a thirty-line class shrinking to about five lines with `@Data`.

EMPHASISE

Flag the IDE gotcha so they don't lose time in the lab: Lombok works through annotation processing. If the IDE shows 'cannot find method `getTitle`', they simply need to enable annotation processing in IntelliJ, or install the Lombok plugin in Eclipse. The code is correct - the IDE just hasn't been told to run Lombok yet.

Part 2 - Exception Handling

Slide 10 - What happens without custom exception handling?

ON SCREEN

Two mock responses: an ugly 500 with a stack trace, versus a clean 404 with a clear message.

SAY (read aloud)

In Session 2, asking for a missing id just returned null or a blank body. If your code throws instead, Spring's default is an ugly, leaky error. Look at the left mock: a 500 status - wrong, because it's not the server's fault - with a long stack trace that leaks your internals to the client. Now the right mock: a clean 404 Not Found, with a clear human message, 'Post not found with id equals 5', and no internals leaked. That right-hand response is the goal. When a record isn't found, the client should get a 404 with a helpful message - not a 500 with a wall of stack trace. We get there with a custom exception, which is the next slide.

Slide 11 - Custom exception, how it works

ON SCREEN

A three-step visual, then the `ResourceNotFoundException` class and the `orElseThrow` line that throws it.

SAY (read aloud)

Three small pieces turn an ugly 500 into a clean 404. Step one: create a class that extends `RuntimeException` and takes a message in its constructor, passing it to `super`. Step two: annotate that class with `@ResponseStatus` of `HttpStatus` dot `NOT_FOUND` - Spring reads this and sets the response code to 404 automatically. Step three: throw it, using `orElseThrow`, when a record is missing. Look at the code: the `ResourceNotFoundException` class is just a few lines - the annotation plus a constructor calling `super` with the message. And below it, the service throws it with `findById` dot `orElseThrow`, passing a new `ResourceNotFoundException` whose message includes the id. No try-catch, no extra config - the annotation does the wiring.

Slide 12 - Optional and `orElseThrow()`

ON SCREEN

A two-path diagram from `findById` to `present-or-empty`, then the `orElseThrow` code, and a warning about `.get()`.

SAY (read aloud)

When you call `findById`, Spring Data returns an `Optional` of `Post` - a container that may or may not hold a value. It exists to force you to handle the not-found case, preventing the dreaded `NullPointerException`. Follow the two paths: `findById` gives an `Optional`; if it's present, you get the `Post`; if it's empty, you throw the `ResourceNotFoundException`. The code reads almost like English: 'find by id, or else throw this exception.' If found, the `post` variable holds it; if not, the exception is thrown and Spring turns it into a 404.

EMPHASISE

Warn against the common mistake: don't call `get` directly on an `Optional` - if it's empty it throws a generic `NoSuchElementException` with no useful message. Always prefer `orElseThrow`, so you control both the exception and the message the client receives.

Part 3 - Spring Security & Microservices (overview)

Slide 13 - Spring Security: what it gives you

ON SCREEN

Three cards - blocks everything, login page, generated password - plus the three override properties.

SAY (read aloud)

Now a first taste of Spring Security - this is a demo, not a lab. Just by adding one dependency to the classpath, Spring Security locks down your entire application with sensible defaults, before you write a single line of security code. Three things happen automatically. It blocks everything - every endpoint now requires authentication. It provides a login page in the browser, with no HTML written by you, defaulting the username to 'user'. And it prints a randomly generated password in the console at startup, which you use to log in the first time. You override these with three lines in `application.properties` - a username, a password, and you're set.

EMPHASISE

Set expectations clearly: this slide is a demo walkthrough only, no lab exercise. Full security - JWT tokens, role-based access, custom config - is a dedicated later session. Today's point is just how little it takes to switch protection on.

Slide 14 - Monolithic architecture: the problem

ON SCREEN

A diagram of one big application with all modules deployed together, and three pain-point cards.

SAY (read aloud)

To set up microservices, let's first name the thing they replace. Everything we've built so far is a monolith: one application, one deployable unit, all modules bundled together. That's perfectly fine to start with - but as a system grows, the monolith starts to hurt. Three pain points. One failure equals total outage - if the Payments module crashes, it can take the whole application down; there's no isolation. Scale everything or nothing - if only Orders is under heavy load, you still have to duplicate the entire app to scale that one part. And risky deployments - a one-line change to Inventory means rebuilding and redeploying the entire application, so releases get slow and scary.

Slide 15 - Microservices architecture: the solution

ON SCREEN

The same system split into independent services, each with its own database, and three benefit cards.

SAY (read aloud)

Microservices break that one big app into many small, independent services, each owning one business capability and its own database, communicating over the network - usually REST. Compare directly with the monolith. Independent deployment: update the Payment service and deploy just that one - the rest keep running. Independent scaling: if Order traffic spikes, run more copies of only the Order service - pay for what you actually need. And failure isolation: if Payments goes down, Users and Inventory keep serving - one failure no longer means total outage.

DELIVERY TIP

Be honest about the trade-off, as the slide notes: microservices add network calls, distributed data, and operational complexity. They shine for large systems and large teams. A small project is often better as a well-structured monolith. Knowing when to use each is the real skill.

Slide 16 - Horizontal vs vertical scaling

ON SCREEN

Two columns: vertical scaling (one bigger server) versus horizontal scaling (more servers behind a load balancer).

SAY (read aloud)

When traffic grows you have two options. Vertical scaling - scale up - means making a single machine more powerful, more RAM and CPU. It's simple, but there's a hard ceiling - you can't add RAM forever - and big machines get expensive. It's still one point of failure. Horizontal scaling - scale out - means adding more machines running the same service, behind a load balancer. There's no real ceiling, it's cost-effective with commodity hardware, and it's naturally fault-tolerant.

EMPHASISE

Connect it forward: microservices are designed for horizontal scaling. Because each service is small and stateless, you can spin up as many copies as you need and spread traffic across them - exactly the model the cloud is built for. This sets up Session 4.

Slide 17 - Load balancing & high availability

ON SCREEN

A load-balancer diagram: clients to a load balancer to three instances of a service.

SAY (read aloud)

If you run three copies of a service, something has to decide which copy each request goes to. That's a load balancer - it sits in front and distributes incoming traffic evenly across the instances. Two benefits. Load balancing spreads requests so no single instance is overwhelmed, and it removes the single point of failure. High availability means if one instance dies, the load balancer simply stops sending it traffic and the others keep serving - users never notice. In the Spring world this is handled by Spring Cloud, with Eureka as a registry where services find each other, and an API Gateway as a smart front door. Just names for now - you'll use them in Session 4.

Slide 18 - Full architecture at a glance

ON SCREEN

A microservices diagram with the Post service highlighted as 'what we built' and others greyed out.

SAY (read aloud)

Here's the payoff slide. Everything you've built across Sessions 1 to 3 - the layered Post API - is one service in a picture like this. The greyed-out boxes are future territory; the highlighted one is what you can build today. Clients hit an API Gateway, the single entry point. The gateway routes to the right service - your Post service, and later a User service, a Payment service, and so on. The takeaway: you're not building toy apps - you're building the fundamental unit of a real distributed system. A clean, layered, well-behaved service is exactly what slots into an architecture like this. Master the one service, and the system is just many of them, cooperating.

Recap & Close

Slide 19 - What's coming next

ON SCREEN

Four cards: Spring Security deep-dive, Spring Cloud basics, inter-service communication, database per service.

SAY (read aloud)

Today you saw teasers of Spring Security and microservices. Future sessions turn those previews into hands-on skills. A Spring Security deep-dive: real authentication with JWT tokens and role-based access control. Spring Cloud basics: Eureka so services can find each other, and an API Gateway as the single front door. Inter-service communication: how one microservice calls another, building on the RestTemplate idea from Session 2, plus modern clients like OpenFeign. And database-per-service: why each microservice owns its own data. Everything connects - today's clean layered service is the foundation every one of these builds on.

Slide 20 - Session recap + annotation cheat-sheet

ON SCREEN

Five plain-English concepts at the top, an annotation grid below: @Entity, @Data, @Repository, @Service, @ResponseStatus, @Autowired.

SAY (read aloud)

Let's recap five concepts in plain English. A DTO is a plain class that carries only what the client needs, keeping the entity hidden. Layered architecture is Controller to Service to Repository, each with one job. A custom exception is your own RuntimeException with a meaningful message. At-ResponseStatus maps that exception to the right HTTP code, like 404, automatically. And orElseThrow is the clean way to throw when an Optional is empty - no null checks. The annotation grid is your revision anchor: at-Entity marks a database table, at-Data generates the boilerplate, at-Repository marks the data layer, at-Service marks the business layer, at-ResponseStatus sets the status code, and at-Autowired injects a dependency.

Slide 21 - Closing

ON SCREEN

The closing blue slide: 'Let's build it.'

SAY (read aloud)

That's the theory for Session 3. You can now explain why we split an app into layers, why we never expose entities, and how custom exceptions give clients clean errors. Now let's open the lab and build the layered Post API together - the entity, the DTO, the custom exception, the service with its two mappers, and the controller. By the end you'll have a clean, professional, persistent API. Let's build it.